GIÁO TRÌNH THỰC HÀNH • NODE.JS BACKEND

CHƯƠNG 04
RESTful CRUD & Swagger

Hoàn thiện API Quản lý sinh viên bằng TypeScript và tài liệu OpenAPI YAML riêng.

NODE.JS + EXPRESS + TYPESCRIPT + PNPM

Học Prompt Engineering theo khung CTCO • Thực hành từng bước • Kiểm thử với Swagger & Postman

BÀI THỰC HÀNH BUỔI 5 / 01

Mục tiêu & cách học

Tiếp tục dự án của các buổi trước. Mỗi bước có yêu cầu kỹ thuật, prompt ngắn, thao tác thực hành và tiêu chí kiểm tra.

05	01	12
REST API GET / POST / PUT / DELETE	File OpenAPI YAML mô tả API	Ca kiểm thử có minh chứng

Sau bài thực hành, sinh viên làm được

1. Giải thích luồng **Route → Controller → Service → dữ liệu** và trách nhiệm từng tầng.
2. Hoàn thiện hai chức năng còn thiếu: **PUT** (sửa) và **DELETE** (xóa).
3. Tạo giao diện Swagger UI tại `/api-docs` đọc đặc tả `docs/openapi.yaml`.
4. Kiểm thử thành công và thất bại bằng Swagger, Postman; kiểm chứng dữ liệu sau thao tác ghi.
5. Tự viết prompt theo **CTCO**, đọc code AI tạo ra và ghi nhật ký học tập.

Quy trình làm việc với AI

Đọc yêu cầu → Xác định file → Viết prompt → Đọc code → Build → Test → Giải thích

> **Quan trọng:** Dùng lại project `backend-quan-ly-sinh-vien`; **không khởi tạo project mới**. Nếu đang dùng SQL Server + Sequelize thì giữ kết nối đó. Nếu project đang lưu bằng RAM, tiếp tục RAM và ghi rõ hạn chế dữ liệu mất khi khởi động lại.

Hồ sơ cần nộp

Mã nguồn; file YAML; ảnh Swagger; Postman Collection; bảng kết quả test; `docs/prompt-log.md` và phần giải thích các thay đổi. Không nộp `node_modules` hay mật khẩu trong `.env`.

BÀI THỰC HÀNH BUỔI 5 / 02

Quy ước REST API

Base URL ví dụ: http://localhost:5000. Thay 5000 bằng PORT thực tế của project.

Method	Endpoint	Mục đích / thành công
GET	`/api/v1/sinh-vien`	Danh sách · 200
GET	`/api/v1/sinh-vien/:maSV`	Chi tiết · 200
POST	`/api/v1/sinh-vien`	Thêm mới · 201
PUT	`/api/v1/sinh-vien/:maSV`	Cập nhật · 200
DELETE	`/api/v1/sinh-vien/:maSV`	Xóa · 200

Hợp đồng dữ liệu

POST: bắt buộc đủ `maSV`, `hoTen`, `email`, `maLop`. Mã sinh viên không được trùng.

PUT: lấy `maSV` từ URL; body chỉ gồm `hoTen`, `email`, `maLop` (đủ cả ba). Không đổi `maSV` qua body.

Các giá trị là chuỗi không rỗng sau `trim`; email phải có định dạng cơ bản hợp lệ. Không cho phép trường ngoài hợp đồng. Tôn trọng giới hạn độ dài và khóa ngoại hiện có trong SQL Server nếu đang sử dụng.

Response GET / POST / PUT thành công

```
{ "success": true,
  "message": "...",
  "data": { /* sinh viên */ } }
```

Response lỗi

```
{ "success": false,
  "message": "...",
  "data": null }
```

> **Lưu ý:** GET danh sách luôn trả `data` là mảng (kể cả `[]`). DELETE thành công trong bài **trả 200** với `{"success":true,"message":"Xóa sinh viên thành công"}`, không có `data`. POST trùng mã: 409; dữ liệu sai: 400; mã không tồn tại: 404.

BÀI THỰC HÀNH BUỔI 5 / 03

Cây thư mục & vị trí file

Tên file dưới đây là quy ước thống nhất. Nếu dự án thật có tên khác, giữ tên thật và ghi chú tương ứng.

```
backend-quan-ly-sinh-vien/
├── package.json
├── pnpm-lock.yaml
├── tsconfig.json
├── .env                        # PORT, DB (không chia sẻ mật khẩu)
├── .env.example                # Các khóa cấu hình mẫu
├── src/
│   ├── server.ts               # Express, mount router, listen
│   ├── config/
│   │   ├── env.config.ts       # Nếu đã có
│   │   ├── database.ts         # Nếu dùng SQL Server
│   │   └── swagger.config.ts   # MỚI: đọc YAML
│   ├── routes/
│   │   └── sinhVien.routes.ts
│   ├── controllers/
│   │   └── sinhVien.controller.ts
│   ├── services/
│   │   └── sinhVien.service.ts
│   ├── dtos/
│   │   └── sinhVien.dto.ts     # Nếu project có
│   ├── models/
│   │   └── sinhVien.model.ts   # Nếu dùng Sequelize
│   └── middlewares/
│       └── error.middleware.ts # Nếu project có
├── docs/
│   ├── openapi.yaml            # MỚI: đặc tả OpenAPI
│   ├── prompt-log.md           # MỚI: nhật ký prompt
│   └── test-results.md         # MỚI: nhật ký kiểm thử
├── postman/
│   └── Buoi5-CRUD.postman_collection.json
└── dist/                       # Tự sinh khi build
```

> **Đường đi:** `server.ts` gắn router → `routes` khai báo phương thức và URL → `controllers` nhận/trả HTTP → `services` xử lý nghiệp vụ và nguồn dữ liệu.

Nếu project có `app.ts`: cấu hình `app.use(...)` tại đó, còn `server.ts` chỉ chạy lắng nghe. Không tạo hai Express app hoặc hai bộ Route/Controller trùng chức năng.

BÀI THỰC HÀNH BUỔI 5 / 04

Bước 0: Kiểm tra project trước khi sửa

Mở VS Code tại thư mục có package.json. Chạy lệnh riêng từng dòng.

```
node --version
pnpm --version
pnpm install
pnpm build
pnpm dev
```

Nếu chưa có `pnpm`, chạy `npm install -g pnpm` rồi kiểm tra `pnpm --version`. Nếu thiếu script `dev` hoặc `build`, xem `package.json` của bài trước; không tự đoán cách chạy.

Đối chiếu dự án hiện tại

1. Xem `package.json` (scripts, module) và `tsconfig.json` (module, moduleResolution).
2. Tìm PORT trong `.env`; xem app đọc biến này ở file nào. Không gửi mật khẩu .env cho AI.
3. Kiểm tra ba API nền: **GET danh sách**, **GET theo mã**, **POST**.
4. Nếu GET/POST chưa chạy được: sửa chúng trước rồi mới thêm PUT/DELETE.

GỢI Ý PROMPT • 0 · Đọc hiểu project

Tôi đang tiếp tục project quản lý sinh viên bằng Express và TypeScript.
Hãy đọc các file Route, Controller, Service, server/app, package.json và tsconfig.json mà tôi cung cấp.
1. Chỉ ra đường đi của request GET.
2. Xác định file cần sửa để thêm PUT và DELETE.
3. Chưa viết code và không tạo project mới.
Trình bày thành bảng: file – trách nhiệm – việc cần làm.

Sinh viên tự điền: File gắn router: _____ • File nhận HTTP: _____ • File nghiệp vụ: _____ • Nguồn dữ liệu: _____

KỸ NĂNG AI / PROMPT ENGINEERING / 05

Học viết Prompt Engineering theo CTCO

Prompt không cần dài. Hãy mô tả đủ thông tin quan trọng và chia tác vụ thành các bước nhỏ.

C	T	C	O
CONTEXT	**TASK**	**CONSTRAINT**	**OUTPUT**
Bối cảnh	Nhiệm vụ	Ràng buộc	Đầu ra

Thành phần	Tự hỏi trước khi gửi AI
Context	Tôi đang làm dự án gì? Công nghệ và file hiện có là gì?
Task	Tôi muốn AI làm **một việc cụ thể** nào?
Constraint	Điều gì bắt buộc? Điều gì AI không được tự đổi?
Output	Tôi muốn nhận code, bảng phân tích hay hướng dẫn kiểm tra?

Từ yêu cầu chưa rõ → prompt dễ thực hiện

Chưa tốt: "Viết giúp tôi toàn bộ Backend CRUD thật chuẩn, đầy đủ mọi thứ."
Thiếu bối cảnh, đường dẫn file, phạm vi và tiêu chí đạt.

GỢI Ý PROMPT • Ví dụ CTCO

Tôi đang làm Backend quản lý sinh viên bằng Express + TypeScript.
Hãy bổ sung chức năng PUT /api/v1/sinh-vien/:maSV.
Yêu cầu: giữ kiến trúc Route → Controller → Service, không đổi nguồn dữ liệu, trả 400 khi body sai và 404 khi mã không tồn tại.
Chỉ ra file cần sửa, code thay đổi và cách kiểm thử.

Hai nguyên tắc học với AI: (1) Mỗi prompt giải quyết một tác vụ. (2) Sau khi AI trả lời, sinh viên phải đọc, chạy và xác nhận kết quả thật.

Có thể bổ sung **Role** (vai trò) hoặc **Example** (ví dụ mẫu) khi cần, nhưng không bắt buộc phải viết nhiều nhãn và ràng buộc lặp lại trong mọi prompt.

BÀI THỰC HÀNH BUỔI 5 / 06

Bước 1: Bổ sung API PUT

Nhiệm vụ: cập nhật thông tin sinh viên hiện có, không tạo bản ghi mới.

Request cần thực hiện

```
PUT http://localhost:5000/api/v1/sinh-vien/SV003
Content-Type: application/json

{
  "hoTen": "Ngọc Mai Updated",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

File	Chức năng cần bổ sung
`routes/sinhVien.routes.ts`	Khai báo PUT theo `:maSV`.
`controllers/sinhVien.controller.ts`	Đọc params/body, gọi Service, trả response.
`services/sinhVien.service.ts`	Kiểm tra, tìm và cập nhật dữ liệu.

GỢI Ý PROMPT • 1 · Cập nhật PUT

Tôi đang làm dự án quản lý sinh viên bằng Express và TypeScript theo kiến trúc Route → Controller → Service.
Hãy bổ sung PUT /api/v1/sinh-vien/:maSV.
1. Lấy maSV từ URL; body có đủ hoTen, email, maLop.
2. Body sai trả 400, không có mã trả 404, thành công trả 200.
3. Giữ nguồn dữ liệu và kiến trúc hiện tại.
Chỉ ra file cần sửa, cung cấp code và giải thích luồng xử lý.

Kiểm tra sau khi thêm

1. Tạo trước `SV003` bằng POST (nếu chưa tồn tại).
2. PUT body hợp lệ → 200; GET lại → dữ liệu thực sự thay đổi.
3. PUT thiếu `email` → 400; GET lại → dữ liệu cũ phải giữ nguyên.
4. PUT mã không tồn tại → 404; không sinh thêm bản ghi.

Cần hiểu: `maSV` nằm ở `req.params`, thông tin cập nhật ở `req.body`. Service không nhận trực tiếp `req/res` của Express.

BÀI THỰC HÀNH BUỔI 5 / 07

Bước 2: Bổ sung API DELETE

Nhiệm vụ: xóa một sinh viên theo maSV, sau đó chứng minh bản ghi đã bị xóa.

Request

```
DELETE http://localhost:5000/api/v1/sinh-vien/
SV003

// Không gửi body.
```

Response thành công

```
{
  "success": true,
  "message": "Xóa sinh viên thành công"
}
```

Đường đi cần triển khai

Route: khai báo `router.delete('/:maSV', handler)`. **Controller:** lấy mã, gọi Service và trả status. **Service:** tìm bản ghi và xóa từ RAM hoặc SQL Server theo kiến trúc cũ.

GỢI Ý PROMPT • 2 · Xóa DELETE

Tiếp tục project trên. Hãy bổ sung DELETE /api/v1/sinh-vien/:maSV.
1. Lấy maSV từ URL và xóa qua Service hiện có.
2. Không tìm thấy trả 404; xóa thành công trả 200 với thông báo.
3. Không thay đổi các API khác và không tạo nguồn dữ liệu mới.
Cho tôi code cần sửa theo đúng vị trí file và cách test bằng Postman.

Thử nghiệm theo thứ tự

Lần gọi	Thao tác	Kết quả đúng
1	DELETE SV003 đã tạo để thử	200
2	GET SV003 sau khi xóa	404
3	DELETE SV003 lần nữa	404

An toàn dữ liệu: Chỉ xóa mã thử nghiệm. Nếu SQL Server báo lỗi khóa ngoại, không tự tắt ràng buộc hoặc xóa dây chuyền; hãy dùng bản ghi thử không có quan hệ liên quan.

Tự trả lời: Vì sao DELETE lần thứ hai trả 404 là kết quả đúng? Vì sao phải GET lại sau khi xóa?

BÀI THỰC HÀNH BUỔI 5 / 08

Bước 3: Kiểm tra đủ 5 API CRUD

Sinh viên tự soát Route, Controller và Service trước khi chuyển sang Swagger.

```
// src/routes/sinhVien.routes.ts - minh họa
router.get('/', getDanhSachSinhVien);
router.get('/:maSV', getSinhVienTheoMa);
router.post('/', taoSinhVien);
router.put('/:maSV', capNhatSinhVien);
router.delete('/:maSV', xoaSinhVien);

// Trong server.ts hoặc app.ts
app.use('/api/v1/sinh-vien', sinhVienRoutes);
```

Tên handler chỉ là ví dụ; tên import/export phải đúng project thật. Không tạo thêm router chưa được gắn vào Express.

Chuỗi kiểm tra có ý nghĩa

Thứ tự	Method	Điều phải chứng minh
1	GET list	Trả data là mảng (có thể rỗng).
2	POST	SV003 mới được tạo, trả 201.
3	GET detail	Đọc thấy SV003 đã lưu.
4	PUT	Sửa thông tin; GET lại đúng giá trị mới.
5	DELETE	Xóa bản ghi; GET lại 404.

> **Đừng nhầm:** HTTP 200/201 chỉ cho biết kết quả phản hồi; **GET lại** giúp kiểm tra dữ liệu có thực sự được lưu/cập nhật/xóa, thay vì server trả JSON mẫu.

Câu hỏi ôn tập

1. Tại sao có hai API GET nhưng đều thuộc Read?
2. Nếu dùng GET thay PUT trong Postman, có kích hoạt được handler PUT không?
3. Validation nằm ở đâu? Ai quyết định HTTP status code?
4. Dữ liệu RAM và SQL Server khác nhau thế nào sau khi restart?

BÀI THỰC HÀNH BUỔI 5 / 09

Bước 4: Swagger với file YAML riêng

Swagger UI hiển thị tài liệu và cho phép thử API. YAML mô tả hợp đồng, không tự sinh logic Backend.

TypeScript (.ts)

Thực thi Backend: Route, Controller, Service, loader và server.

OpenAPI YAML (.yaml)

Mô tả đường dẫn, tham số, requestBody, schema, response.

Cài thư viện trong project hiện tại

```
pnpm add swagger-ui-express js-yaml
pnpm add -D @types/swagger-ui-express @types/js-yaml openapi-types
```

File	Trách nhiệm
`docs/openapi.yaml`	Toàn bộ tài liệu API OpenAPI 3.0.3.
`src/config/swagger.config.ts`	Đọc file YAML bằng `js-yaml`, xuất swaggerSpec.
`src/server.ts` hoặc `app.ts`	Mount giao diện tại `/api-docs`.
`src/routes/sinhVien.routes.ts`	Giữ định tuyến; **không** thêm comment `@swagger`.

Nếu từng dùng JSDoc: Chỉ duy trì **một nguồn mô tả API**: `docs/openapi.yaml`. Không sử dụng `swagger-jsdoc` để quét chú thích trong bài này. Chỉ gỡ dependency cũ khi xác nhận không còn được dùng.

Nhớ cấu trúc YAML

```yaml
openapi: 3.0.3
info:
  title: API Quản lý sinh viên
  version: 1.0.0
servers:
  - url: /
paths:
  /api/v1/sinh-vien:
    get:
      responses:
        '200':
          description: Thành công
```

Khung trên chỉ để hiểu cú pháp; file đầy đủ có trong **Phụ lục A**. Dùng dấu cách (không dùng Tab); dùng `{maSV}` trong OpenAPI, còn Express Route dùng `:maSV`.

BÀI THỰC HÀNH BUỔI 5 / 10

Bước 5: Tạo docs/openapi.yaml

AI hỗ trợ tạo tài liệu, nhưng sinh viên phải đối chiếu với API và quy ước dữ liệu trang 3.

GỢI Ý PROMPT • 3 · Đặc tả YAML

Dự án Express + TypeScript của tôi đã có 5 API CRUD sinh viên.
Hãy tạo docs/openapi.yaml chuẩn OpenAPI 3.0.3.
1. Mô tả GET list, GET detail, POST, PUT, DELETE.
2. Có tham số maSV, requestBody, schema và status code phù hợp.
3. Tái dùng components/$ref; không dùng JSDoc hoặc decorator.
Trả về file YAML hoàn chỉnh và giải thích những phần quan trọng.

Đọc hiểu các khóa trong OpenAPI

Khóa	Ý nghĩa
`openapi`, `info`	Phiên bản đặc tả và thông tin API.
`servers`	Nơi gửi request; `url: /` dùng cùng host/PORT.
`paths`	Các endpoint với method GET/POST/PUT/DELETE.
`requestBody`	Dữ liệu JSON client phải gửi.
`responses`	Status code và dữ liệu phản hồi.
`components`, `$ref`	Định nghĩa chung để tái sử dụng schema, tham số, lỗi.

Những điểm phải tự kiểm tra

1. **POST:** body đủ bốn trường (bao gồm `maSV`) và status 201/400/409.
2. **PUT:** body đủ ba trường được sửa, `maSV` chỉ nằm ở URL; status 200/400/404.
3. **DELETE:** thành công trả `200` với `success` và `message` (không có `data`).
4. **GET danh sách:** `data` là mảng. Response lỗi thống nhất `success:false`, `data:null`.
5. Các tham chiếu `$ref` phải trỏ đến schema/response có thật.

Phân biệt: Có `required` trong YAML chỉ giúp mô tả dữ liệu bắt buộc. Backend vẫn phải tự kiểm tra request khi chạy.

BÀI THỰC HÀNH BUỔI 5 / 11

Bước 6: Tích hợp Swagger bằng TypeScript

Dùng file YAML riêng, giữ ứng dụng Express hiện tại và không tạo server thứ hai.

GỢI Ý PROMPT • 4 · Tích hợp Swagger UI

Tôi đã có file docs/openapi.yaml trong dự án Express + TypeScript.
Hãy tích hợp Swagger UI tại /api-docs.
1. Tạo src/config/swagger.config.ts đọc YAML bằng js-yaml.
2. Sử dụng swagger-ui-express trong server/app đang có.
3. Giữ nguyên Route, Controller, Service và cách import của dự án.
Chỉ rõ vị trí file và đoạn code cần bổ sung.

File src/config/swagger.config.ts (mẫu)

```
import { readFileSync } from 'node:fs';
import { resolve } from 'node:path';
import { load } from 'js-yaml';
import type { OpenAPIV3 } from 'openapi-types';

const yamlPath = resolve(process.cwd(), 'docs/openapi.yaml');
const value: unknown = load(readFileSync(yamlPath, 'utf8'));
const isObj = (v: unknown): v is Record<string, unknown> =>
  typeof v === 'object' && v !== null && !Array.isArray(v);

if (!isObj(value) || value.openapi !== '3.0.3' ||
    !isObj(value.info) || !isObj(value.paths)) {
  throw new Error('OpenAPI YAML thiếu cấu trúc bắt buộc');
}
export const swaggerSpec = value as OpenAPIV3.Document;
```

Trong file server.ts hoặc app.ts đang có

```
import swaggerUi from 'swagger-ui-express';
import { swaggerSpec } from './config/swagger.config.js';

// Đặt sau khi đã tạo app; trước handler 404 / lỗi.
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

Đuôi `.js` trong import nội bộ áp dụng cho project ESM/NodeNext như bài mẫu. Nếu project có quy ước khác, dùng quy ước đang hoạt động. Ép kiểu TypeScript không có nghĩa đã validate toàn bộ đặc tả ở runtime.

BÀI THỰC HÀNH BUỔI 5 / 12

Bước 7: Kiểm thử bằng Swagger UI

Swagger giúp đọc và thử request trực tiếp trên trình duyệt, nhưng không tự đảm bảo nghiệp vụ đúng.

Khởi động và mở tài liệu

```
pnpm build
pnpm dev
# Mở: http://localhost:5000/api-docs
```

1. Thấy nhóm **SinhVien** với đủ năm operation.
2. Mở **GET /api/v1/sinh-vien** → Try it out → Execute: xem status **200** và data dạng mảng.
3. Mở **POST**, nhập JSON mẫu bên dưới: mong đợi **201**; tiếp tục GET mã đó để xác nhận.
4. Mở **PUT**, nhập `maSV=SV003` và body mới: mong đợi **200**; GET lại để kiểm tra đã cập nhật.
5. Mở **DELETE**, xóa mã thử `SV003`: mong đợi **200**; GET lại mã đó phải **404**.

JSON để POST

```
{
  "maSV": "SV003",
  "hoTen": "Ngọc Mai",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

JSON để PUT

```
{
  "hoTen": "Ngọc Mai Updated",
  "email": "ngocmai@example.com",
  "maLop": "25CT114"
}
```

Cố tình kiểm thử trường hợp lỗi

Trường hợp	Status mong đợi
PUT thiếu `email`	400
GET mã không tồn tại	404
POST lại `maSV` đã tồn tại	409

> **Lưu ý dữ liệu mẫu:** Dùng `25CT114` chỉ khi mã lớp hợp lệ trong project. Nếu dùng SQL Server có khóa ngoại, chọn mã lớp đang có thật.

BÀI THỰC HÀNH BUỔI 5 / 13

Bước 8: Làm quen Postman

Cùng một REST API nhưng Postman giúp lưu request thành Collection để kiểm thử lặp lại.

Thiết lập Collection

1. Mở Postman → tạo Collection **Buoi5 - CRUD SinhVien**.
2. Trong Collection, tạo biến `baseUrl` = `http://localhost:5000` (hoặc PORT thật).
3. Tạo request mới, chọn Method phù hợp, nhập URL, bấm Save rồi Send.

Tên request	Method và URL
01 GET list	GET `{{baseUrl}}/api/v1/sinh-vien`
02 POST	POST `{{baseUrl}}/api/v1/sinh-vien`
03 GET detail	GET `{{baseUrl}}/api/v1/sinh-vien/SV003`
04 PUT	PUT `{{baseUrl}}/api/v1/sinh-vien/SV003`
05 GET verify	GET `{{baseUrl}}/api/v1/sinh-vien/SV003`
06 DELETE	DELETE `{{baseUrl}}/api/v1/sinh-vien/SV003`
07 GET missing	GET `{{baseUrl}}/api/v1/sinh-vien/SV003`

Nhập body đúng cách

Với POST và PUT: vào **Body → raw → JSON**; dán JSON ở trang trước. Kiểm tra header `Content-Type: application/json`. Với DELETE: để Body là **none**.

> **Cách đọc kết quả:** Ở Response, xem **Status** và **Body**. Trả 400 hoặc 404 không có nghĩa bài test hỏng, nếu bạn đang chủ động thử dữ liệu sai hoặc mã không tồn tại.

> **Nếu Postman Web không gọi được localhost:** sử dụng Postman Desktop hoặc Desktop Agent đã kết nối. Không cần đưa Backend lên Internet để vượt lỗi này.

BÀI THỰC HÀNH BUỔI 5 / 14

Bước 9: Viết assertion trong Postman

Một test đạt khi kết quả thực tế khớp kết quả mong đợi: status, body và trạng thái dữ liệu.

Ví dụ: kiểm tra POST thành công

Trong request POST, mở **Scripts → Post-response** (hoặc Tests tùy giao diện Postman), dán đoạn script:

```
pm.test('POST trả 201', () => {
  pm.response.to.have.status(201);
});
pm.test('Sinh viên mới đúng dữ liệu', () => {
  const json = pm.response.json();
  pm.expect(json.success).to.eql(true);
  pm.expect(json.data.maSV).to.eql('SV003');
  pm.expect(json.data.hoTen).to.eql('Ngọc Mai');
});
```

Ví dụ: xác nhận PUT có hiệu lực

Gửi PUT rồi mở request GET SV003, đặt script kiểm tra dữ liệu mới:

```
pm.test('GET sau PUT trả dữ liệu mới', () => {
  pm.response.to.have.status(200);
  const json = pm.response.json();
  pm.expect(json.data.hoTen).to.eql('Ngọc Mai Updated');
  pm.expect(json.data.email).to.eql('ngocmai@example.com');
});
```

Ví dụ: xác nhận DELETE đã xóa

```
pm.test('GET sau DELETE trả 404', () => {
  pm.response.to.have.status(404);
  const json = pm.response.json();
  pm.expect(json.success).to.eql(false);
});
```

> **Tại sao phải GET lại?** Một API có thể trả JSON đúng dù không ghi dữ liệu. GET sau POST/PUT/DELETE giúp phát hiện trường hợp này. Nếu dùng SQL Server, còn có thể restart server rồi GET để kiểm tra dữ liệu được lưu bền vững.

BÀI THỰC HÀNH BUỔI 5 / 15

Bảng 12 test case bắt buộc

Chạy theo thứ tự. Trước khi bắt đầu, đảm bảo SV003 chưa tồn tại và SV9999 là mã không có trong dữ liệu.

ID	Điều kiện / thao tác	Status	Xác nhận thêm
T01	GET danh sách	200	`data` là mảng
T02	POST SV003 hợp lệ, chưa tồn tại	201	GET SV003 đọc được
T03	POST SV003 lần hai	409	Chỉ có một SV003
T04	POST mã mới thiếu hoTen	400	Không tạo bản ghi
T05	GET SV003 sau tạo	200	Đúng cả bốn trường
T06	PUT SV003 đủ ba trường	200	GET lại đúng giá trị mới
T07	PUT SV003 thiếu email	400	Dữ liệu cũ không đổi
T08	PUT SV9999 body hợp lệ	404	Không tạo mã mới
T09	GET SV9999	404	`data:null`
T10	DELETE SV003	200	`success:true`
T11	GET SV003 sau xóa	404	Sinh viên không còn
T12	DELETE SV003 lần hai	404	Không ảnh hưởng mã khác

Ghi kết quả trong docs/test-results.md

Với mỗi test: ghi **ID, dữ liệu ban đầu, method/URL, body, status mong đợi, status thực tế, response, kết quả GET kiểm chứng, PASS/FAIL, ảnh/log**. Nếu FAIL, ghi nguyên nhân và kết quả sau sửa; không đổi expected để che lỗi.

> **Ví dụ T07:** Trước PUT, SV003 có email A. Gửi PUT thiếu email → 400; GET lại thấy email A và tên không bị đổi. Kết luận PASS vì yêu cầu sai đã bị từ chối và dữ liệu được bảo toàn.

BÀI THỰC HÀNH BUỔI 5 / 16

Bước 10: Ghi prompt log & tự viết prompt

Mục đích không phải chứng minh đã hỏi AI, mà chứng minh sinh viên hiểu yêu cầu, chỉnh sửa và kiểm tra được sản phẩm.

GỢI Ý PROMPT • 5 · Nhờ AI gợi ý test

Tôi đã có 5 API CRUD sinh viên trong Postman.
Hãy đề xuất các test case thành công và thất bại, có status mong đợi.
Yêu cầu: kiểm tra POST trùng mã, PUT thiếu email, DELETE rồi GET lại; không sửa Backend.
Trả về bảng test gồm thao tác, dữ liệu đầu vào, kết quả mong đợi và cách xác nhận.

Mẫu docs/prompt-log.md

```
# Prompt log - Buổi 5
## Nhiệm vụ: Bổ sung PUT
- Bối cảnh và file tôi đã đưa cho AI: ...
- Prompt do tôi viết: ...
- AI đề xuất thay đổi: ...
- File / đoạn code tôi đã đọc và sửa: ...
- Build thực tế: ...
- Test T06/T07/T08: expected ...; actual ...
- Cách tôi xác nhận dữ liệu: ...
- Tôi giải thích được điều gì: ...
- Vấn đề còn lại: ...
```

Bài tự viết prompt (không sao chép): Hãy dùng CTCO để yêu cầu AI bổ sung tính năng tìm sinh viên theo lớp. Sinh viên tự xác định endpoint đề xuất, file phải chỉnh sửa, ràng buộc không làm hỏng CRUD cũ và cách kiểm thử. Trước khi code, giải thích vì sao chọn endpoint đó.

Điểm giảng viên có thể hỏi

1. Trong prompt của bạn, đâu là Context? Task? Constraint? Output?
2. AI đã đổi những file nào? Vì sao không nên viết truy vấn DB ở Route?
3. Mã HTTP 400, 404, 409 khác nhau ra sao?
4. Test nào chứng minh dữ liệu đã ghi thật, không chỉ trả response đẹp?

BÀI THỰC HÀNH BUỔI 5 / 17

Xử lý lỗi khi thực hành

Dùng thông báo lỗi và file liên quan để hỏi AI; không yêu cầu AI "viết lại toàn bộ dự án".

Hiện tượng	Cách tự kiểm tra đầu tiên
`pnpm build` lỗi import/export	Tên handler có khớp? Module NodeNext có cần `.js` trong import?
`req.body` rỗng	Body raw JSON; `Content-Type`; `express.json()` đặt trước router.
PUT/DELETE luôn 404	Đúng method, URL, tiền tố `app.use` và mã SV có tồn tại?
Swagger "No operations defined"	`openapi.yaml` có `paths`? Đọc đúng file? Restart server?
`ENOENT` không thấy YAML	Đúng đường dẫn `docs/openapi.yaml`? Chạy từ thư mục gốc?
`YAMLException`	Kiểm tra khoảng trắng, dấu nháy, thụt dòng; không dùng Tab.
POST 201 nhưng GET không thấy	Service có thực hiện lưu? Đã `await`? Có dùng cùng nguồn dữ liệu?
Restart mất dữ liệu	RAM sẽ mất; nếu SQL Server, kiểm tra kết nối và lệnh ghi.
DELETE lỗi khóa ngoại	Dùng bản ghi thử không có quan hệ; giữ ràng buộc dữ liệu.

Prompt sửa lỗi có dữ kiện

GỢI Ý PROMPT • 6 · Debug

Tôi đang làm bước [tên bước] của project Express + TypeScript.
Lệnh/request: [...]. Mong đợi: [...]. Thực tế và log lỗi: [...].
Đây là file liên quan: [...].
Hãy giải thích nguyên nhân, chỉ ra sửa đổi tối thiểu và cách kiểm tra lại.
Không đổi kiến trúc, endpoint hoặc nguồn dữ liệu hiện tại.

Bảo mật: Không đưa mật khẩu SQL Server, JWT token, thông tin cá nhân hay toàn bộ `.env` vào chat. Nếu dùng AI agent trong IDE, kiểm tra thay đổi (diff) trước khi chấp nhận. Nếu dùng chat thường, bạn phải tự gửi file liên quan; AI không tự đọc được ổ đĩa máy bạn.

BÀI THỰC HÀNH BUỔI 5 / 18

Bài tập mở rộng: Bài viết & lượt xem

Tự vận dụng CTCO để mô tả một nghiệp vụ mới mà không thay đổi chức năng SinhVien.

Yêu cầu nghiệp vụ

Tài nguyên **BaiViet** có `id`, `title`, `content`, `view_count`.

API	Ý nghĩa
GET `/api/v1/bai-viet/:id`	Đọc bài viết, **không** tăng view_count.
POST `/api/v1/bai-viet/:id/views`	Ghi nhận lượt xem, view_count tăng đúng 1.

Mã bài không tồn tại → 404. Ghi nhận thành công → 200 và trả về `view_count` mới. Nếu lưu bằng SQL Server/ Sequelize, cần tăng nguyên tử để hạn chế mất lượt khi nhiều request cùng lúc.

GỢI Ý PROMPT • Mở rộng · Tự phân tích trước

Tôi muốn thêm chức năng Bài viết trong dự án Express + TypeScript.
GET /api/v1/bai-viet/:id chỉ đọc; POST /api/v1/bai-viet/:id/views tăng view_count đúng 1.
Hãy phân tích file Route, Controller, Service và dữ liệu cần có; đề xuất cách kiểm thử.
Chưa viết code. Giữ nguyên các chức năng CRUD sinh viên.

Kiểm thử cần tự thiết kế

1. GET bài viết đang có → số lượt xem không đổi.
2. POST ghi nhận lần 1 → tăng 1; GET xác nhận. POST lần 2 → tăng thêm 1.
3. POST mã không tồn tại → 404; không làm thay đổi bài viết khác.

Vì sao không dùng GET để tăng view_count? GET dùng để truy xuất tài nguyên; cache, robot, prefetch và retry có thể tạo các lượt GET không phải hành động ghi nhận lượt xem có chủ đích. Ví dụ bài này tách rõ thao tác đọc và thao tác ghi.

BÀI THỰC HÀNH BUỔI 5 / 19

Checklist nộp bài & ôn tập cuối buổi

Chỉ đánh dấu hoàn thành khi đã chạy thực tế và có minh chứng.

Checklist kỹ thuật

☐ `pnpm build` không lỗi; `pnpm dev` chạy đúng PORT.
☐ Đủ GET list, GET detail, POST, PUT, DELETE.
☐ Route, Controller, Service đúng trách nhiệm; không tạo server mới.
☐ PUT nhận đủ ba trường; DELETE 200 có thông báo.
☐ `/api-docs` đọc đúng `docs/openapi.yaml` và hiển thị năm API.
☐ Đã chạy 12 test, ghi actual, PASS/FAIL và ảnh/log.
☐ Có GET xác minh sau POST, PUT và DELETE.
☐ Có Postman Collection, prompt log và giải thích code bằng lời của mình.
☐ Ghi rõ project dùng RAM hay SQL Server; không nộp bí mật.

Cấu trúc minh chứng gợi ý

```
docs/
├── openapi.yaml
├── prompt-log.md
├── test-results.md
└── screenshots/
    ├── swagger-operations.png
    ├── post-created.png
    ├── put-verified.png
    └── deleted-get-404.png
postman/
└── Buoi5-CRUD.postman_collection.json
```

Những câu hỏi cần trả lời được

PUT đi qua những file nào? Vì sao GET sau DELETE trả 404 là test đạt? Swagger YAML có tự xử lý nghiệp vụ không? Vì sao phải GET lại sau thao tác ghi? Sau restart, dữ liệu có còn không? CTCO giúp prompt rõ ràng như thế nào?

> **Tham khảo:** tài liệu thực hành Buổi 5 phiên bản trước; BÀI THỰC HÀNH 5 (phong cách prompt chia tác vụ); tài liệu Express, OpenAPI Specification 3.0, js-yaml, swagger-ui-express và Postman.

MÃ NGUỒN THAM KHẢO / A1

Phụ lục A: File openapi.yaml — phần 1/4

Các dòng 1–48 của cùng một file docs/openapi.yaml. Nối liên tiếp các phần A1 → A4, không chép tiêu đề trang.

```yaml
openapi: 3.0.3
info:
  title: API Quản lý sinh viên
  version: 1.0.0
  description: RESTful CRUD SinhVien - thực hành Buổi 5.
servers:
  - url: /
    description: Cùng host và port với Express
tags:
  - name: SinhVien
    description: Quản lý sinh viên
paths:
  /api/v1/sinh-vien:
    get:
      tags: [SinhVien]
      summary: Lấy danh sách sinh viên
      operationId: getDanhSachSinhVien
      responses:
        '200':
          description: Thành công, có thể trả danh sách rỗng
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SinhVienListResponse'
        '500':
          $ref: '#/components/responses/InternalError'
    post:
      tags: [SinhVien]
      summary: Thêm sinh viên mới
      operationId: taoSinhVien
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SinhVien'
      responses:
        '201':
          description: Tạo sinh viên thành công
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SinhVienResponse'
        '400':
          $ref: '#/components/responses/BadRequest'
        '409':
          $ref: '#/components/responses/Conflict'
        '500':
```

Đầu file phải có `openapi: 3.0.3` và `paths` chỉ khai báo một lần.

MÃ NGUỒN THAM KHẢO / A2

Phụ lục A: File openapi.yaml — phần 2/4

Các dòng 49–95 của cùng một file docs/openapi.yaml. Nối liên tiếp các phần A1 → A4, không chép tiêu đề trang.

```
        $ref: '#/components/responses/InternalError'
  /api/v1/sinh-vien/{maSV}:
    parameters:
      - $ref: '#/components/parameters/MaSV'
    get:
      tags: [SinhVien]
      summary: Lấy sinh viên theo mã
      operationId: getSinhVienTheoMa
      responses:
        '200':
          description: Tìm thấy sinh viên
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SinhVienResponse'
        '404':
          $ref: '#/components/responses/NotFound'
        '500':
          $ref: '#/components/responses/InternalError'
    put:
      tags: [SinhVien]
      summary: Cập nhật thông tin sinh viên
      operationId: capNhatSinhVien
      description: Gửi đủ hoTen, email, maLop; maSV lấy từ URL.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/SinhVienUpdate'
      responses:
        '200':
          description: Cập nhật thành công
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/SinhVienResponse'
        '400':
          $ref: '#/components/responses/BadRequest'
        '404':
          $ref: '#/components/responses/NotFound'
        '500':
          $ref: '#/components/responses/InternalError'
    delete:
      tags: [SinhVien]
      summary: Xóa sinh viên theo mã
      operationId: xoaSinhVien
```

Tiếp nối trực tiếp phần trước. Giữ đúng thụt dòng và phân biệt chữ hoa/chữ thường trong `$ref`.

MÃ NGUỒN THAM KHẢO / A3

Phụ lục A: File openapi.yaml — phần 3/4

Các dòng 96–141 của cùng một file docs/openapi.yaml. Nối liên tiếp các phần A1 → A4, không chép tiêu đề trang.

```
      responses:
        '200':
          description: Xóa thành công
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/DeleteResponse'
        '404':
          $ref: '#/components/responses/NotFound'
        '500':
          $ref: '#/components/responses/InternalError'
components:
  parameters:
    MaSV:
      name: maSV
      in: path
      required: true
      schema:
        type: string
        minLength: 1
      example: SV003
  schemas:
    SinhVien:
      type: object
      additionalProperties: false
      required: [maSV, hoTen, email, maLop]
      properties:
        maSV: {type: string, minLength: 1, example: SV003}
        hoTen: {type: string, minLength: 1, example: Ngọc Mai}
        email: {type: string, format: email, example: ngocmai@example.com}
        maLop: {type: string, minLength: 1, example: 25CT114}
    SinhVienUpdate:
      type: object
      additionalProperties: false
      required: [hoTen, email, maLop]
      properties:
        hoTen: {type: string, minLength: 1, example: Ngọc Mai Updated}
        email: {type: string, format: email, example: ngocmai@example.com}
        maLop: {type: string, minLength: 1, example: 25CT114}
    SinhVienResponse:
      type: object
      required: [success, message, data]
      properties:
        success: {type: boolean, enum: [true], example: true}
        message: {type: string, example: Thao tác thành công}
        data:
```

Tiếp nối trực tiếp phần trước. Giữ đúng thụt dòng và phân biệt chữ hoa/chữ thường trong `$ref`.

MÃ NGUỒN THAM KHẢO / A4

Phụ lục A: File openapi.yaml — phần 4/4

Các dòng 142–190 của cùng một file docs/openapi.yaml. Nối liên tiếp các phần A1 → A4, không chép tiêu đề trang.

```
          $ref: '#/components/schemas/SinhVien'
    SinhVienListResponse:
      type: object
      required: [success, message, data]
      properties:
        success: {type: boolean, enum: [true], example: true}
        message: {type: string, example: Lấy danh sách thành công}
        data:
          type: array
          items:
            $ref: '#/components/schemas/SinhVien'
    DeleteResponse:
      type: object
      required: [success, message]
      properties:
        success: {type: boolean, enum: [true], example: true}
        message: {type: string, example: Xóa sinh viên thành công}
    ErrorResponse:
      type: object
      required: [success, message, data]
      properties:
        success: {type: boolean, enum: [false], example: false}
        message: {type: string, example: Yêu cầu không thực hiện được}
        data: {type: object, nullable: true, example: null}
  responses:
    BadRequest:
      description: Dữ liệu đầu vào không hợp lệ
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    NotFound:
      description: Không tìm thấy sinh viên
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    Conflict:
      description: Mã sinh viên đã tồn tại
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    InternalError:
      description: Lỗi ngoài dự kiến trên server
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
```

Tiếp nối trực tiếp phần trước. Giữ đúng thụt dòng và phân biệt chữ hoa/chữ thường trong `$ref`.